

Can PlanetSimulation run in a browser?

Two-page feasibility memo · 2 October 2026 · native v0.0.1 at c464cd3

Assessment: feasible with a dedicated browser platform and graphics adapter. The C++ simulation, noise, camera math and astronaut IK can largely be reused. The current executable cannot be ported by changing the compiler alone: its desktop OpenGL features, blocking frame loop and filesystem workflow need adaptation. A WebGL 2 demonstration is the smaller first step; WebGPU is the stronger candidate if preserving compute-based foliage and the planned GPU terrain is the priority. This is a source audit and design assessment, not a measured browser build or an FPS prediction.

Compilation and delivery

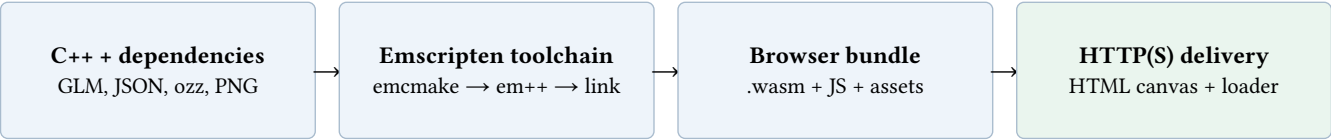


Figure 1: Proposed build: compile native libraries for Wasm, then package shaders and scenes. GPU shaders are compiled by the browser, not into the C++ Wasm module.

Emscripten supplies the CMake toolchain through emcmake [S1]. A new CMake platform branch must replace the present required system OpenGL, GLEW and GLFW packages; native static libraries cannot be linked into Wasm. Pin the SDK and library versions, compile C++20 dependencies for that target, and explicitly preserve the application’s config exceptions. Package shaders/configs for the virtual filesystem or fetch them before initialization. Ship optimized Wasm with JavaScript glue and an HTML shell; use a local HTTP server for development.

Current dependency	Proposed browser equivalent / remaining work
GLM + nlohmann JSON	Reuse headers and CPU double-precision coordinates; verify unit tests under Wasm.
ozz-animation runtime	Cross-compile the pinned IK runtime. Its source provides a scalar SIMD reference path; validate gait before optional Wasm SIMD tuning.
GLFW + GLEW	Use an Emscripten GLFW canvas port [S2] or HTML5 bindings. Replace desktop GL loading and context hints with GLES/WebGL capability checks.
libpng + shader files	Cross-compile PNG or export through browser APIs; package GLSL libraries and preserve their inclusion order.
std::async + diagnostics	Choose worker-backed threads or incremental CPU jobs; replace OS telemetry and native capture/input automation.

The renderer needs more than a fallback switch

The desktop 4.3 foliage path uses compute shaders, SSBOs and indirect draws. Core WebGL 2 lacks those APIs [S3]. Disabling compute still leaves `samplerBuffer` terrain vertices, `usamplerBuffer` indices and the trail BVH buffer texture in the 3.3 vertex path. Replace those with bounded 2-D data textures and integer `texelFetch` addressing, or a revised vertex attribute representation. Revalidate roots, LODs, trail deformation and transfer costs. Convert 330 core shaders to 300 es, add precision declarations, and audit formats, sampler limits and extensions [S3, S4]. Existing instanced draws, shadow maps and fragment atmosphere passes provide useful reusable structure.

Runtime, tradeoffs and a practical first port

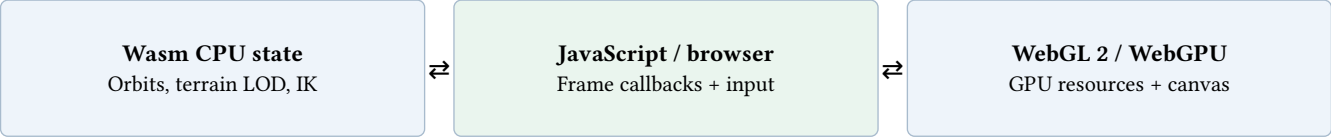


Figure 2: Proposed runtime boundary. Workers may prepare terrain; virtual files hold assets, while optional IndexedDB holds persistent state. Browser callbacks own frame scheduling.

Caveat	Required adaptation and runtime dependency
Frame loop / input	Split Interactive.cpp’s blocking while loop into persistent state and one-frame callbacks via emscripten_set_main_loop / requestAnimationFrame [S5]. Canvas focus, resize, pointer lock and keyboard capture need browser handling.
Terrain workers	Threaded std::async needs a pthread build, SharedArrayBuffer and cross-origin isolation with COOP/COEP headers [S6]. Serve securely (localhost for development). Keep a separate single-thread build with incremental terrain work; avoid main-thread waits.
Files / replay	MEMFS is temporary; IDBFS persists through IndexedDB synchronization [S7]. Replace filesystem timestamp watching with explicit config reload/import, and PNG/JSON writes with downloads. Persistence remains versioned and quota-limited.
HDR / readback	Require EXT_color_buffer_float for RGBA16F/RG32F render targets [S8]. Current RG32F linear lookup also needs float-linear filtering or manual interpolation. Replace stencil/depth readback with colour-encoded diagnostics; use supported RGBA float readback for the highlight meter.
Profiling / recovery	NVML GPU utilization/VRAM is unavailable through the browser adapter. Track owned allocations; time passes only when timer extensions are available and results are non-disjoint [S9]. Rebuild resources after context loss.

Benefits and costs

Benefits: a shareable URL removes native installation for demos and lets the same C++ simulation support browser and desktop frontends. JSON scenes and replays remain useful exchange formats. Static hosting simplifies delivery; the browser sandbox confines access to the host filesystem.

Costs: a second renderer/platform path needs continued testing. Asset download, startup compilation, memory limits, browser/GPU variation and tab throttling change the experience. Native llvmpipe timings do not predict browser FPS. The existing grass queues alone reserve about 25.6 MB at 200,000 candidates; terrain, water and HDR reflections add further allocations. Set measured browser budgets; offline 20× foliage is not a suitable default mobile workload.

Recommendation and acceptance gates

First isolate the window, frame lifecycle, assets and diagnostics; retain the native tests. Then build a WebGL 2 slice with terrain, atmosphere, water, astronaut and rewritten vertex-path foliage. Require correct shadows/trails, config import/export, context recovery and responsive terrain updates. Measure frame-time percentiles, peak owned memory and download/startup size in Chromium, Firefox and Safari on actual target devices. Use tolerant image comparisons across drivers, while checking deterministic simulation and same-backend replay.

If those budgets cannot support the required density, evaluate a separate WebGPU renderer: compute/storage buffers match the current GPU design [S10], but bindings, pipelines and GLSL-to-WGSL shaders must be rewritten. Emscripten provides a C API route through Emdawnwebgpu [S11]. Probe capabilities at runtime; neither route guarantees current desktop quality or performance without a port.

Primary documentation checked 2 October 2026. Project-specific conclusions are inferred from the audited source; links describe APIs, not measured project performance.

S1 — CMake builds · S2 — GLFW canvas port · S3 — WebGL 2 · S4 — GL mapping · S5 — frame callbacks · S6 — threads · S7 — storage · S8 — float targets · S9 — GPU timing · S10 — WebGPU compute · S11 — WebGPU C API · float filtering.